

Dokumentace projektu z předmětů IFJ a IAL
**Implementace překladače imperativního
jazyka IFJ23**

Tým xsedot00, varianta vv-BVS

Rozšíření: **FUNEXP**

Dias Assatulla (xassat00) – 25 %

David Kvaček (xkvace00) – 25 %

Jakub Sychra (xsychr06) – 25 %

Tomáš Šedo (xsedot00) – 25 %

Brno, 6. prosince 2023

Obsah

1	Úvod	2
2	Návrh a implementace	3
2.1	Lexikální analýza	3
2.2	Syntaktická analýza	3
2.2.1	Zpracování výrazů	3
2.3	Sémantická analýza	4
2.4	Generování mezikódu	4
2.4.1	Začátek generování	4
2.4.2	Generování proměnných	4
2.4.3	Generování výrazů	4
2.4.4	Generování funkcí	5
2.4.5	Generování IF a WHILE	5
2.5	Tabulka symbolů	5
3	Rozšíření	6
3.1	FUNEXP	6
4	Týmová spolupráce	7
4.1	Rozdělení práce	7
4.2	Verzovací systém	7
4.3	Komunikace	7
5	Závěr	8
6	Přílohy	9
6.1	Diagram konečného automatu lexikálního analyzátoru	9
6.2	Precedenční tabulka syntaktického analyzátoru	10
6.3	LL(1) gramatika syntaktického analyzátoru	11
7	Literatura	12

1 Úvod

V rámci zadání je úkolem vyvinout překladač pro jazyk IFJ23, který převádí zdrojový kód napsaný v tomto jazyce do cílového jazyka IFJcode23 (mezikód). Jazyk IFJ23 je zjednodušenou podmnožinou jazyka Swift vyvíjeného společností Apple. Překladač má být implementován v programovacím jazyce C a je určen k běhu v konzolovém prostředí bez grafického uživatelského rozhraní. Jeho hlavním úkolem je provádět lexikální analýzu, syntaktickou analýzu a sémantickou analýzu zdrojového kódu, přičemž detekuje a oznamuje různé druhy chyb.

Návratová hodnota překladače je definována podle typu chyby, která nastane během překladu. Pokud proběhne překlad bez chyb, program vrací hodnotu 0. V opačném případě jsou chyby kategorizovány a návratová hodnota je stanovena podle povahy chyby, od chybné struktury lexikálního tokenu až po sémantické nekonzistence v programu.

Implementace překladače zahrnuje načítání řídicího programu ze standardního vstupu a generování mezikódu na standardní výstup. Chybová hlášení, varování a ladicí výpisy jsou směřovány na standardní chybový výstup, což umožňuje uživateli efektivní sledování průběhu překladu.

Interpretace výsledného mezikódu IFJcode23 je realizována pomocí interpretu, který je k dispozici na stránkách předmětu. Překladač je navržen s ohledem na komplexní zpracování zdrojového kódu a poskytuje uživateli informace o případných chybách a varováních v rámci překladu.

2 Návrh a implementace

Překladač pro jazyk IFJ23 je elegantně navržen a implementován tak, aby efektivně zpracovával zdrojový kód. Jeho struktura reflektuje logiku lexikální analýzy, syntaktické analýzy a sémantické analýzy, přičemž každá část je implementována v samostatném souboru. Tyto části vzájemně komunikují pomocí jasně definovaných rozhraní, což zajišťuje koherentní chod celého programu.

Lexikální analyzátor identifikuje tokeny a předává je syntaktické analýze, která provádí strukturální kontrolu a sémantickou analýzu. Pokud zdrojový kód úspěšně projde všemi kontrolami, je spuštěna fáze generování výsledného mezikódu. Soubory obsahující implementaci jednotlivých částí jsou systematicky uspořádány, což usnadňuje správu a údržbu kódu^[2].

2.1 Lexikální analýza

Hlavní funkcí lexikálního analyzátoru (LA) je generování tokenů, které slouží k následnému zpracování ve fázi syntaktické analýzy. Tuto úlohu plní funkce `get_token()`, jež čte postupně znaky ze standardního vstupu nebo ze vstupního souboru.

Před implementací lexikálního analyzátoru byl vytvořen deterministický konečný automat (DKA)¹. Na základě tohoto automatu bylo realizováno chování lexikálního analyzátoru pomocí opakujícího se switch-case, kde každý case odpovídá jednomu stavu automatu.

Tato metodika umožňuje systematicky zpracovat vstupní sekvenci a identifikovat lexémy. Každý stav automatu reprezentuje specifický stav analýzy. Přístup s deterministickým konečným automatem poskytuje robustní a efektivní mechanismus pro analýzu a identifikaci tokenů, což je klíčový krok v procesu kompilace programovacího jazyka IFJ23.

2.2 Syntaktická analýza

Syntaktická analýza získává tokeny z lexikální části programu prostřednictvím funkce `nextToken()`. Tato funkce je důkladně využívána v různých částech aplikace pro správný průchod zdrojovým kódem, který se řídí LL gramatikou^{6.3}. Dále je schopná identifikovat příznak `\n`, což je klíčové pro rozpoznání korektní struktury programu v kontextu odřádkování.

Logika syntaktické analýzy spočívá v metodě rekurzivního sestupu, která následuje definovaná pravidla gramatiky a řídí se typem zpracovávaného tokenu. Po každém úspěšném zpracování pravidla se vytvoří a následně uloží nová položka do abstraktního syntaktického stromu (AST). Tento strom je dále využíván při generování cílového kódu.

Pro korektní průběh analýzy je nezbytné dodržování syntaktických pravidel daných gramatikou. Každé zpracované pravidlo přispívá k vytváření struktury AST, což následně umožňuje efektivní generování cílového kódu v souladu s definovanými gramatickými pravidly.

2.2.1 Zpracování výrazů

Pro zpracování výrazů jsou využívány potřebné funkce implementované v souboru `expressions.c`. Proces zahrnuje načtení dvou tokenů, přičemž první token je pečlivě kontrolován, aby bylo ověřeno, zda je vhodný pro použití ve výrazu. Následně se druhý token používá k identifikaci případů, kdy se jedná o volání funkce ve výrazu. Tento případ má speciální zpracování a výsledek je následně vrácen do zpracování výrazu, kde se chová jako identifikátor, přičemž detaily volání jsou uloženy ve vlastní struktuře.

Každý token je vložen na zásobník a nad ním jsou prováděny operace podle precedenční tabulky^{6.2}. Tento přístup umožňuje efektivní zpracování výrazů s ohledem na prioritní operace a správné zjištění výsledku.

Zpracování výrazů je klíčovým prvkem v rámci syntaktické a sémantické analýzy, a jeho implementace v souboru `expressions.c` poskytuje efektivní mechanismus pro manipulaci s tokeny a operacemi ve výrazech.

2.3 Sémantická analýza

Účelem sémantické analýzy je dedukovat význam programu, který je vyjádřen jeho syntaktickou strukturou. K ověření, zda má daná syntaktická struktura smysl, se využívá tabulka symbolů (TS), dvousměrně vázaný seznam (DLL) a abstraktní syntaktický strom (AST).

V případě identifikace nového rámce sémantická analýza inicializuje lokální tabulku symbolů, začlení ji do dvousměrně vázaného seznamu a aktivuje ji. Pokud sémantická analýza potřebuje získat určité informace o daném identifikátoru, vyhledávání probíhá očekávaným způsobem, tj. postupně od aktuálně aktivního rámce až po zcela globální rámec celého programu.

Když je syntaktická struktura správná, volá se jedna z funkcí: `semantic_return()`, `semantic_handler()` nebo `semantic_expression()` v závislosti na typu abstraktního syntaktického stromu. Funkce `semantic_return()` slouží k ověření typu návratové hodnoty funkce, zatímco `semantic_handler()` kontroluje proměnnou s explicitní deklarací typu. V případě, že typ není uveden, ale existuje hodnota této proměnné nebo výraz, funkce `semantic_expression()` provádí příslušné kontroly.

Tyto funkce dále kontrolují:

- existenci dané proměnné,
- možnost opakované definice (v případě nemodifikovatelné proměnné `let`),
- redeklaraci proměnné ve stejném rámci,
- a další sémantické aspekty specifikované v IFJ23.

2.4 Generování mezikódu

Pojmem generátor mezikódu se rozumí proces generování mezikódu IFJcode23 na standardní výstup ve formě jednotlivých instrukcí. Mezikód je následně interpretován, pokud je kód syntakticky správný. Pro generování mezikódu je využit abstraktní syntaktický strom (AST), který je definován v syntaktické analýze. Generátor je implementován jako samostatný modul v souboru `generator.c` společně s přidruženým hlavičkovým souborem `generator.h`.

2.4.1 Začátek generování

Na začátku programu jsou definovány proměnné, které jsou používány v průběhu celého programu, a je definován začátek generování v rámci funkce `initGenerator()`. Tato funkce je volána v syntaktické analýze (přesněji v souboru `parser.c`). V této funkci se získává globální tabulka symbolů a vytváří se pomocný dvousměrně vázaný seznam pro lokální tabulky symbolů. Současně je generována hlavička, která musí být na začátku mezikódu, a to řetězec `.IFJcode23`. Po vygenerování hlavičky začíná samotný proces generování jednotlivých struktur prostřednictvím funkce `generateCode()`.

2.4.2 Generování proměnných

Proměnné mohou být definovány i bez deklarace. V případě, že proměnná má datový typ umožňující vložit hodnotu `nil` (`Double?`, `Int?` nebo `String?`) a současně není deklarována, je do této proměnné implicitně uložena právě hodnota `nil`. Proměnné lze definovat dvěma způsoby: pomocí `let` (dále již nemodifikovatelná), nebo `var` (běžná definice).

2.4.3 Generování výrazů

Výrazy jsou generovány převážně pomocí zásobníkových instrukcí, pokud jsou dostupné. Výsledek po provedení instrukce je uložen do příslušné proměnné. V případě nezásobníkových instrukcí jsou výsledky výrazů uloženy přímo do proměnné bez potřeby pracovat se zásobníkem. U komplexnějších výrazů jsou operace prováděny podle priority definované zadáním.

2.4.4 Generování funkcí

Při generování funkcí se využívá dočasného rámce, v němž je vytvořena lokální tabulka symbolů, do níž jsou ukládány všechny proměnné příslušící dané funkci. Po skončení funkce je lokální tabulka symbolů odstraněna. Pro ukládání této tabulky se využívá pomocného seznamu. Funkce mohou být definovány s parametry, nebo zcela bez parametrů. Při volání funkce jsou parametry uloženy na zásobník a postupně přiřazovány každému parametru funkce.

2.4.5 Generování IF a WHILE

Ve většině případů je využíváno generování prvků již zmíněných výše. Návěští mají speciální názvy, například pro IF: `ifxelse` a `ifxelseend`, kde `x` je číslo inkrementující se s každým dalším vytvořeným IF. Stejný princip platí také pro WHILE: `whilex` a `whilexend`. Pro skoky na tato návěští jsou využívány podmíněné skoky, především s využitím zásobníkových funkcí.

2.5 Tabulka symbolů

Na základě vybrané varianty implementace zadání byla tabulka symbolů navržena a implementována jako binární vyhledávací strom s výškovou vyvážeností. Hlavním účelem této implementace je systematické uchovávání zpracovaných identifikátorů, což usnadňuje jejich následné vyhledávání. Každý uzel tabulky obsahuje informace o svém typu, rámci, povaze (proměnná či funkce), definici, možnosti modifikace, počtu parametrů funkce, ukazateli na lokální tabulku symbolů dané funkce a další.

Binární vyhledávací strom zajišťuje efektivní a rychlé vyhledávání identifikátorů v tabulce symbolů. Kořeny jednotlivých tabulek symbolů jsou organizovány v pomocném dvousměrně vázaném seznamu, což umožňuje snadný přístup k jednotlivým tabulkám. Po prvotní inicializaci globální tabulky symbolů dochází k implicitnímu naplnění vestavěnými funkcemi a klíčovými slovy. Dále, pokud je to nutné, se pro každý nový rámec, který se v načteném vstupním programu vyskytne, inicializuje samostatná lokální tabulka symbolů pro daný rámec a následně se vloží do dvousměrně vázaného seznamu, ze kterého se po skončení tohoto rámce opět odstraní.

V souboru `symtable.h` jsou deklarovány jednotlivé funkce a výčtové typy, zatímco implementace tabulky symbolů a příslušné operace se nachází v souboru `symtable.c`. Tímto způsobem je dosaženo systematické správy identifikátorů a efektivního vyhledávání v rámci komplexního zpracování zdrojového kódu^[1].

3 Rozšíření

Při implementaci syntaktického analyzátoru byla vzata do úvahy realizace případných rozšíření, které obsahovalo samotné zadání projektu. Po zvážení náročnosti jednotlivých rozšiřujících součástí a aktuálního stavu projektu bylo implementováno jediné následující rozšíření.

3.1 FUNEXP

Během procesu zpracování jednotlivých výrazů v rámci funkcí implementovaných ve zdrojovém souboru `expressions.c` se načítají dva po sobě jdoucí tokeny získané z lexikálního analyzátoru. Právě zmíněný druhý token v pořadí slouží ke zjištění všech případů, kdy se skutečně jedná o volání funkce v momentálně zpracovávaném výrazu.

Zpracování takového výrazu je pochopitelně odlišné od toho obvyčejného. Nejdříve se vyhodnotí samotné volání funkce a poté se tento zjištěný výsledek vrací do zpracovávaného výrazu. Samotné detaily volání této funkce jsou uchovávány ve své vlastní struktuře. Výsledek se následně v tomto výrazu chová jako obvyčejný identifikátor a zpracování takového výrazu je poté již klasické.

4 Týmová spolupráce

Týmová spolupráce při implementaci překladače pro jazyk IFJ23 byla důležitým prvkem vlastní realizace aplikace. Každý člen týmu přinesl své unikátní dovednosti a perspektivu, což umožnilo efektivní rozdělení úkolů a řešení komplexních výzev. Pravidelné komunikační kanály a společné schůzky byly rozhodující pro synchronizaci práce, výměnu nápadů a řešení případných problémů. Důkladné testování a vzájemná revize kódu napomohly k dosažení konzistentní kvality a funkcionality každé části překladače.

4.1 Rozdělení práce

- Dias Assatulla – lexikální analýza, sémantická analýza, testování,
- David Kvaček – tabulka symbolů, dokumentace, testování,
- Jakub Sychra – syntaktická analýza, abstraktní datové typy, testování,
- Tomáš Šedo – vedoucí týmu, generování mezikódu, testování.

4.2 Verzovací systém

Pro verzování projektu byl zvolen nástroj git s centrálním repozitářem hostovaným na platformě GitHub. Tato kombinace umožňuje efektivní správu a sledování změn v kódu. Git poskytuje robustní nástroje pro větvení, slučování a sledování historie verzí, zatímco GitHub nabízí přehledné rozhraní. Díky této konfiguraci mohou členové týmu jednoduše spolupracovat na projektu, synchronizovat své postupy a udržovat konzistentní a aktualizovaný kód. Git a GitHub jsou významnými nástroji pro moderní vývojové prostředí, které usnadňují spolupráci a správu verzí v softwarovém vývoji.

4.3 Komunikace

Členové týmu navázali spojení při prvotním osobním setkání na zdejší fakultě, což poskytlo příležitost k osobnímu seznámení a vytvoření silnějšího týmového pouta. Pro další pravidelnou komunikaci a schůzky byla zvolena moderní platforma Discord. Tato volba umožňuje efektivní virtuální setkávání včetně hlasových a textových kanálů pro různá témata. Discord rovněž poskytuje okamžitou komunikaci, což usnadňuje řešení otázek a koordinaci úkolů. Tým využívá tuto platformu nejen k pracovním záležitostem, ale také k udržení přátelské atmosféry, která podporuje produktivitu a týmovou spolupráci i v prostředí vzdálené práce.

5 Závěr

Výstupem a konečným výsledkem tohoto zadání projektu je cílový program, který je schopen v určité míře načíst, zpracovat, vyhodnotit a interpretovat vstupní zdrojový kód v jazyce IFJ23 takovým způsobem, že generuje výsledný mezikód IFJcode23. Zpočátku se zdál být tento projekt svým rozsahem velmi náročný, ale po prvotním rozdělení práce a také postupným nabýváním nemalého objemu informací z přednášek tohoto kursu se práce na aplikaci vyvíjela v rámci možností správným směrem.

Ačkoliv práce na tomto projektu zabrala nemalé množství času, každý člen týmu si prakticky vyzkoušel implementaci nějaké části celého překladače. Koneckonců, jedná se ve výsledku o poměrně složitý program složený z několika podstatných modulů, které společně mezi sebou komunikují.

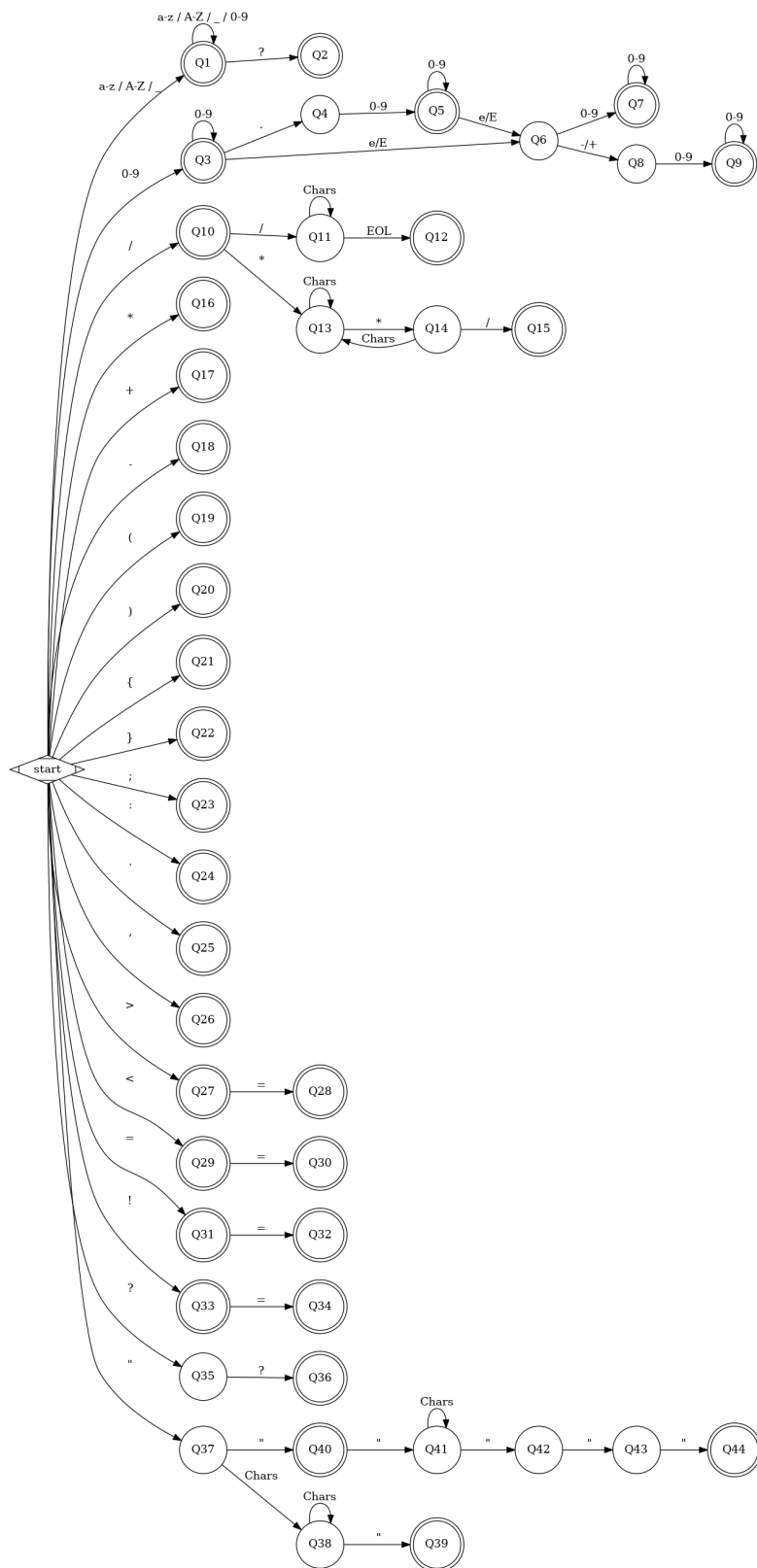
Velkou výhodou je v tomto ohledu možnost pokusného odevzdání, které slouží ke kontrole a ověření již implementované funkcionality. Na základě získaného přibližného hodnocení je následně tým schopen momentální řešení posunout na vyšší úroveň.

Při vývoji takto rozsáhlé aplikace je nutné projít všemi fázemi od návrhu, samotného řešení až po testování. V průběhu realizace výsledného programu se objevilo několik problémů souvisejících nejen s návrhem, ale také s rozhraním mezi jednotlivými moduly. Většina rozsáhlejších problémů se podařila zdárně vyřešit, nicméně na závěr je potřeba dodat, že samotný překladač s jistotou nepracuje bezchybně.

I přesto se sluší říci, že praktické osvojení principů vyučovaných v rámci kursů IFJ a IAL každý člen týmu v budoucnu jistě ocení, ať už se bude jednat o práci v týmu, či samotné implementace takto rozsáhlého problému.

6 Přílohy

6.1 Diagram konečného automatu lexikálního analyzátoru



Obrázek 1: Diagram konečného automatu lexikálního analyzátoru

6.2 Precedenční tabulka syntaktického analyzátoru

	+	-	*	/	==	!=	<	>	<=	>=	ID	??	!	\$
+	>	>	<	<	>	>	>	>	>	>	<	>	<	>
-	>	>	<	<	>	>	>	>	>	>	<	>	<	>
*	>	>	>	>	>	>	>	>	>	>	<	>	<	>
/	>	>	>	>	>	>	>	>	>	>	<	>	<	>
==	<	<	<	<							<	<	<	>
!=	<	<	<	<							<	<	<	>
<	<	<	<	<							<	<	<	>
>	<	<	<	<							<	<	<	>
<=	<	<	<	<							<	<	<	>
>=	<	<	<	<							<	<	<	>
ID	>	>	<	<	>	>	>	>	>	>		>	>	>
??	<	<	<	<	>	>	>	>	>	>	<	<	<	>
!	>	>	>	>	>	>	>	>	>	>		>	<	>
\$	<	<	<	<	<	<	<	<	<	<	<	<	<	

Tabulka 1: Precedenční tabulka syntaktického analyzátoru

6.3 LL(1) gramatika syntaktického analyzátoru

01) <start>	-> <program> <end>
02) <end>	-> eps
03) <end>	-> EOF
04) <program>	-> <statement> <program>
05) <program>	-> func FID (<params>) <func_type> { <statement_block> } <program>
06) <program>	-> eps
07) <statement_block>	-> <statement> <statement_block>
08) <statement_block>	-> eps
09) <func_type>	-> <type>
10) <func_type>	-> eps
11) <params>	-> param_ID ID : <type> <params_n>
12) <params>	-> eps
13) <params_n>	-> , param_ID ID : <type> <params_n>
14) <params_n>	-> eps
15) <type>	-> Int
16) <type>	-> Double
17) <type>	-> String
18) <statement>	-> return expression
19) <statement>	-> <keyword> ID <var_type> <defvar>
20) <statement>	-> while (expression) { <statement_block> }
21) <statement>	-> if <if_type> { <statement_block> } else { <statement_block> }
22) <statement>	-> <rvalue>
23) <statement>	-> ID = <rvalue>
24) <rvalue>	-> FID (<args>)
25) <rvalue>	-> expression
26) <args>	-> param_ID : <term> <args_n>
27) <args>	-> <term> <args_n>
28) <args>	-> eps
29) <args_n>	-> , <args_n_final>
31) <args_n>	-> eps
32) <args_n_final>	-> param_ID : <term> <args_n>
32) <args_n_final>	-> <term> <args_n>
33) <term>	-> ID
34) <term>	-> Int_Value
35) <term>	-> String_Value
36) <term>	-> Double_Value
37) <term>	-> nil
38) <if_type>	-> (expression)
39) <if_type>	-> let ID
40) <defvar>	-> = <rvalue>
41) <defvar>	-> eps
42) <keyword>	-> var
43) <keyword>	-> let
44) <var_type>	-> : <type>
45) <var_type>	-> eps

7 Literatura

- [1] Burgetová, I.; Hranický, R.; Dolejška, D.; aj.: Snímky k přednáškám IAL. online, 2022, [citováno 2023-10-25]. Dostupné z: <https://moodle.vut.cz/mod/folder/view.php?id=223249>
- [2] Meduna, A.; Křivka, Z.: Snímky k přednáškám IFJ. online, 2017-09-17, [citováno 2023-10-07]. Dostupné z: <https://www.fit.vutbr.cz/study/courses/IFJ/public/materials/>